

ПЛАН РАЗРАБОТКИ

Локальная система управления знаниями
на базе OpenKB и GLM-4.7-Flash

Проект	OpenKB GUI — Desktop-приложение для управления базой знаний
Тип	Desktop Application (Python + CustomTkinter)
Язык	Python 3.10+
GUI	CustomTkinter
Количество этапов	7 этапов

Содержание

1. **Обзор проекта** — Цели и задачи системы
2. **Архитектурные принципы** — Философия OpenKB, ключевые отличия от RAG
3. **Этап 1: Инфраструктура** — Подготовка окружения и базовая структура проекта
4. **Этап 2: Конфигурация** — Сервисы настроек и управление workspace
5. **Этап 3: Build Service** — Компиляция базы знаний и интеграция OpenKB
6. **Этап 4: Watch Mode** — Автоматическое отслеживание изменений
7. **Этап 5: Wiki Browser** — Просмотр и навигация по wiki
8. **Этап 6: Chat Module** — Интеллектуальный диалог с базой знаний
9. **Этап 7: Health & Lint** — Диагностика и мониторинг состояния
10. **Сводная таблица этапов** — Хронология и зависимости
11. **Риски и митигация** — Потенциальные проблемы и решения
12. **Заключение** — Итоги и следующие шаги

1. Обзор проекта

1.1 Назначение системы

Проект представляет собой desktop-приложение на Python, которое служит графической оболочкой (GUI) для системы OpenKB. Основная цель — создание инструмента для построения и сопровождения локальной компилируемой базы знаний, работающей по архитектуре OpenKB. Система предназначена для преобразования неструктурированных документов в interlinked markdown wiki с автоматическим синтезом concept pages, summaries и поддержкой knowledge accumulation.

1.2 Ключевые функции

- Импорт локальных документов (PDF, DOCX, TXT, Markdown)
- Компиляция документов в interlinked markdown wiki
- Автоматический синтез concepts pages и summaries
- Накопление знаний с расширением cross-links
- Интеллектуальный диалог поверх compiled wiki
- Watch mode для автоматического обновления
- Lint/health checking для диагностики состояния
- Интеграция с Obsidian для просмотра wiki

1.3 Технический стек

Компонент	Технология	Назначение
Язык	Python 3.10+	Основной язык разработки
GUI	CustomTkinter	Графический интерфейс пользователя
Knowledge Engine	OpenKB	Компиляция базы знаний
LLM Layer	LiteLLM	Унифицированный API к LLM
Модель	GLM-4.7-Flash	Основная AI-модель
Markdown	tkinterweb	Рендеринг markdown в GUI
Packaging	PyInstaller	Сборка standalone executable

2. Архитектурные принципы

2.1 Философия OpenKB

OpenKB используется как knowledge compiler, evolving wiki system, knowledge graph over markdown и long-term accumulated knowledge storage. Это означает, что система не просто индексирует документы, а компилирует их в связную структуру знаний с автоматическим созданием связей между концептами. Каждая страница wiki может ссылаться на другие через wikilinks, образуя граф знаний, который развивается с каждым новым документом.

2.2 Ключевые отличия от классического RAG

Система принципиально отличается от классических RAG-решений. Вместо векторных баз данных, embeddings retrieval и chunk-based поиска используется подход на основе компилированных wiki pages. Это обеспечивает более точные ответы за счёт работы с целостными концептами, а не фрагментами текста.

Аспект	Классический RAG	OpenKB подход
Хранение	Vector database	Compiled markdown wiki
Поиск	Embeddings similarity	Wiki navigation + concepts
Структура	Chunks без связей	Interlinked knowledge graph
Обновление	Re-index embeddings	Incremental wiki rebuild
Накопление	Добавление chunks	Knowledge accumulation

2.3 Структура workspace

Workspace организован по нативной структуре OpenKB с чётким разделением сырых документов и скомпилированной wiki. Папка raw/ содержит исходные документы пользователя, которые подвергаются обработке. Папка wiki/ содержит сгенерированные страницы: concepts/ для автоматически синтезированных концептов, summaries/ для обобщений, explorations/ для результатов исследований, reports/ для отчётов и sources/ для исходных материалов.

3. Этап 1: Инфраструктура

Этап	1: Инфраструктура и базовая структура
Длительность	3-4 дня
Зависимости	Нет

3.1 Задачи

- Создание файловой структуры проекта согласно ТЗ
- Настройка виртуального окружения Python 3.10+
- Установка зависимостей: openkb, customtkinter, litellm, python-dotenv, tkinterweb, watchdog
- Создание базового main.py с точкой входа
- Разработка main_window.py с главным окном CustomTkinter
- Реализация системы вкладок (7 вкладок по ТЗ)
- Настройка .env файла с параметрами LLM

3.2 Результаты

После завершения этапа будет создана базовая структура проекта с главным окном приложения, системой вкладок и настроенным окружением.

Приложение запускается и отображает пустой интерфейс с placeholder-контентом на каждой вкладке.

4. Этап 2: Конфигурация

Этап	2: Сервисы конфигурации
Длительность	2-3 дня
Зависимости	Этап 1

4.1 Задачи

- Разработка config_service.py для загрузки/сохранения настроек из .env
- Реализация UI вкладки Settings (API Key, Base URL, Model, workspace paths)
- Создание логики валидации API ключей
- Реализация выбора и создания workspace директории
- Настройка PAGEINDEX_API_KEY для OCR

- Сохранение состояния настроек между сессиями

4.2 Результаты

Пользователь может настраивать все параметры системы через UI вкладки Settings. Настройки сохраняются в .env файл и загружаются при старте приложения. Валидация API ключей обеспечивает информативные сообщения об ошибках.

5. Этап 3: Build Service

Этап	3: Build Service — Компиляция базы знаний
Длительность	4-5 дней
Зависимости	Этапы 1, 2

5.1 Задачи

- Разработка `build_service.py` для запуска `openkb build` через `subprocess.Popen`
- Реализация `queue-based stdout reading` для логов в реальном времени
- Создание UI вкладки Build (Build button, Watch button, Stop button, logs, progress)
- Реализация `thread-safe GUI updates` через `after()`
- Обработка документов: PDF, PDF scans, DOCX, TXT, Markdown
- Интеграция `PageIndex` для длинных документов (>20 страниц)
- Fallback mode при отсутствии `PAGEINDEX_API_KEY`

5.2 Ключевые технические решения

Build процесс выполняется в отдельном потоке через `subprocess.Popen`, что предотвращает зависание GUI. Вывод процесса читается через `queue` и отображается в текстовом виджете в реальном времени. Кнопка Stop позволяет прервать процесс компиляции. Прогресс-индикатор отображает текущий статус (`idle/building/watching/linting`).

6. Этап 4: Watch Mode

Этап	4: Watch Mode — Автоматическое отслеживание
Длительность	2-3 дня
Зависимости	Этап 3

6.1 Задачи

- Разработка `watch_service.py` для запуска `openkb watch`
- Мониторинг папки `raw/` на наличие новых документов
- Автоматическое обновление `concepts`, `summaries`, `knowledge graph` при изменениях

- Интеграция с UI вкладки Build (Watch button, status indicator)
- Обработка ошибок watch process
- Корректная остановка watch mode

7. Этап 5: Wiki Browser

Этап	5: Wiki Browser — Просмотр wiki
Длительность	4-5 дней
Зависимости	Этапы 1, 3

7.1 Задачи

- Разработка `wiki_service.py` для работы с wiki-файлами
- Создание UI вкладки Wiki Browser (tree view, markdown preview)
- Интеграция `tkinterweb` для рендеринга markdown
- Реализация переходов по wikilinks `[[Concept Name]]`
- Отображение `concepts pages` и `summaries`
- Реализация поиска по wiki
- Отображение `AGENTS.md` в отдельном редакторе

7.2 Результаты

Пользователь получает полноценный браузер wiki с древовидной структурой файлов, предпросмотром markdown и навигацией по wikilinks. Все сгенерированные страницы (`concepts`, `summaries`, `explorations`) доступны для просмотра.

8. Этап 6: Chat Module

Этап	6: Chat Module — Интеллектуальный диалог
Длительность	5-6 дней
Зависимости	Этапы 2, 3, 5

8.1 Задачи

- Разработка `chat_service.py` для работы с LLM через LiteLLM
- Реализация `reasoning over compiled wiki` (не vector RAG)
- Создание UI вкладки Chat (`sessions panel`, `message history`, `input`, `sources`)
- Разработка `session_service.py` для `persistent sessions`
- Отображение `source pages`, `concepts`, `wiki pages` в ответах
- Сохранение/загрузка/удаление/экспорт chat sessions

- Сохранение explorations (результаты исследований)

8.2 Архитектура чата

Чат работает поверх compiled wiki, используя reasoning over wiki structure.

Ответы формируются на основе concepts pages, summaries, wiki links и knowledge graph. Источники ответов (source pages, concepts, wiki pages) отображаются в sources panel. Сессии сохраняются в папку sessions/ в формате JSON.

9. Этап 7: Health & Lint

Этап	7: Health & Lint — Диагностика
Длительность	2-3 дня
Зависимости	Этапы 3, 5

9.1 Задачи

- Разработка lint_service.py для запуска openkb lint
- Создание UI вкладки Health (lint reports, stale pages, warnings)
- Отображение orphan concepts, stale pages, contradictions
- Обнаружение broken links и compilation warnings
- Интеграция с AGENTS.md editor (редактирование конфигурации)
- Кнопка Open in Obsidian для внешнего просмотра

10. Сводная таблица этапов

Этап	Название	Длительность	Зависимости
1	Инфраструктура	3-4 дня	—
2	Конфигурация	2-3 дня	Этап 1
3	Build Service	4-5 дней	Этапы 1, 2
4	Watch Mode	2-3 дня	Этап 3
5	Wiki Browser	4-5 дней	Этапы 1, 3
6	Chat Module	5-6 дней	Этапы 2, 3, 5
7	Health & Lint	2-3 дня	Этапы 3, 5

Общая длительность проекта: 22-29 рабочих дней (при последовательном выполнении). Возможно сокращение за счёт параллельного выполнения независимых этапов (например, Этап 4 и Этап 5 могут разрабатываться параллельно после завершения Этапа 3).

11. Риски и митигация

Риск	Вероятность	Влияние	Митигация
------	-------------	---------	-----------

OpenKB API изменения	Средняя	Высокое	Абстракция через build_service
GUI зависание при build	Высокая	Среднее	subprocess + threading + after()
OCR fallback при отсутствии API key	Средняя	Низкое	Предупреждение пользователя
Проблемы совместимости с Obsidian	Низкая	Среднее	Тестирование экспорта
Ошибки API LLM	Средняя	Высокое	Retry logic + graceful degradation

12. Заключение

План разработки охватывает все ключевые аспекты создания desktop-приложения для системы управления знаниями на базе OpenKB. Поэтапный подход позволяет контролировать прогресс и обеспечивает возможность раннего тестирования каждого модуля. Критически важным является правильная реализация Build Service (Этап 3), который является ядром всей системы.

После завершения всех этапов система будет представлять собой полнофункциональное desktop-приложение с возможностью импорта документов, компиляции базы знаний, интеллектуального диалога и диагностики состояния. Готовое приложение может быть упаковано в standalone executable через PyInstaller для распространения.